# Ayush Yadav

(513) 461-4375 | aesh.03.23@gmail.com | linkedin.com/in/ayush-yadav-developer | github.com/yadava5 | ayush-yadav.com

## SUMMARY

Software engineer, Computer Science graduate (May 2026). Backend in Python and TypeScript, performance code in Java. A service whose database stops one user from reading another's rows, a parallel gzip engine on JDK 25, and a year of ETL in university IT.

## EDUCATION & AWARDS

**Miami University, Oxford, OH | B.S. in Computer Science, Major GPA 3.65** May 2026

- Dean's List: Fall 2023, Spring 2025, Fall 2025.
- Finalist, MUCAT Design Innovation (Feb 2025): LiDAR visual assistance proposal; $2,500 prototyping grant.

## EXPERIENCE

**ITSM Data Integration Intern | Miami University | Oxford, OH** Jun 2025 – May 2026

- Built a Python ETL pipeline over five years of Oracle Analytics query logs to scope a planned Workday reporting migration to Snowflake, reconciled that output against the raw exports, and found 7 of the 50 subject areas never queried.
- Parsed 1.6M Oracle Analytics queries against the repository catalog, a third of them logging no column, to resolve 40.9M column reads to one physical column each; a further 16.9M stayed unattributed, 14.4M of them matching several candidates.
- Consolidated 3,731 rows from Tableau's REST API and 6,743 from Workday's exports, a record hash dropping 21 exact duplicates, into one 10,453 row inventory of report metadata, searchable in one place where it had been two.
- Built a Tableau dashboard on the ETL feed from a Laravel compliance reporter that reads TeamDynamix, GitLab and an End of Life API, covering 37 months of code compliance across 61 projects, 59 compliant and 2 at end of life.

## PROJECTS

**Applied | Gmail Sync and Application Tracking Service** 2026 – Present

*Solo | getapplied.vercel.app*

- Classified each mail into one of eight categories, from assessment and interview to offer and rejection, by 220 hand written rules in a FastAPI service over 9 tables behind a Next.js front end. 64 of 69 application records were filed straight from the mail and 5 corrected by hand, on the one personal Gmail inbox the rules were tuned on.
- Built the tenant boundary into the database, not the query layer, so one user's rows stay unreadable to another from a query with no WHERE clause: all 9 tables forced under row level security with 35 policies, request identity set per transaction, and the service connecting as a role that cannot bypass it. A drift check reasserts it in production after each migration.
- Rekeyed a global email uniqueness index onto the user and message id pair, closing a defect where a second account with the same message id would have failed its whole sync. Resync is idempotent: six passes over the same three mails leave three records, and the service answers a Gmail rate limit with a 429 and Retry-After rather than retrying past the 60 second budget.

**jetpack | Parallel gzip Compression Engine on JDK 25** 2026

*Solo | jetpack-compress.vercel.app*

- Parallelized gzip compression across 10 cores on JDK 25, where the standard library uses one, raising throughput from 66 to 422 MB/s on a 32 MiB synthetic corpus under a three fork JMH harness, and the output stays a gzip file that gunzip opens.
- Compressed a 4 GiB file and a 64 MiB file inside the same 48 MiB heap, bounding the window to two blocks in flight per core, one virtual thread each, written back in order with each worker's CRC-32 combined into the one stream trailer as it went.
- Vectorized Adler-32 by hand with Java's Vector API to fingerprint files in the CLI, not gzip's CRC-32 trailer. It matches java.util.zip bit for bit and runs 2.8 times the scalar loop at 4.26 GB/s, while the JDK's intrinsic leads at 14.06 GB/s.

**Cadence | Natural Language Calendar and Tasks** 2023 – 2026

*Solo | usecadenceapp.vercel.app*

- Unified 36 route handlers behind one catch all serverless function to keep the whole backend inside Vercel's Hobby cap of 12 functions, rewriting only the routing so that no handler had to be merged or dropped.
- Migrated the suite to Testcontainers so the 15 tenant isolation tests that had only run in CI now start their own postgres:16 on any machine, with CI failing when those suites report zero tests or any skip, since a skip and a pass produce the same green tick.
- Built a four stage parser that turns plain English into events and tasks, chrono for dates, then hashtags, then priority, then named entities, with Google sync one way over a window 30 days back and 90 days ahead.

## TECHNICAL SKILLS

**Languages & Web:** Python, Java, C++, TypeScript, SQL, HTML, CSS, React, Next.js, Tailwind CSS, Node.js, Express, FastAPI, REST APIs

**Machine Learning & AI:** PyTorch, scikit-learn, CatBoost, XGBoost, LightGBM, SetFit, ONNX, SHAP, RAG, OpenAI API, embeddings

**Data & Infrastructure:** PostgreSQL (row level security), Supabase, Snowflake, DuckDB, Parquet, pandas, SQLite, ETL, Tableau, Oracle Analytics, OpenMP, SIMD, Vector API, LangGraph, MCP, Docker, JMH, Git, GitHub Actions (CI/CD), Linux, pytest, Vitest